

Analysis of Neural Models for Sandhi Splitting in Ayurvedic Sanskrit Texts

Abhiram A S

Department of Computer Science
Amrita School of Computing
Amrita Vishwa Vidyapeetham,
Amritapuri, Kollam, India
am.en.u4aie22002@am.students.
amrita.edu

Karthik Narayan C

Department of Computer Science
Amrita School of Computing
Amrita Vishwa Vidyapeetham,
Amritapuri, Kollam, India
am.en.u4aie22028@am.students.
amrita.edu

Harigovind C B

Department of Computer Science
Amrita School of Computing
Amrita Vishwa Vidyapeetham,
Amritapuri, Kollam, India
am.en.u4aie22119@am.students.
amrita.edu

Abstract - Sanskrit Sandhi refers to a phonological change whereby words or morphemes that are next to each other alter their sounds to form compound words in which no explicit word boundaries exist. Although Sandhi is vital for the grammatical and beautiful aspects of Sanskrit, it is quite an obstacle for computational processing of the language. Thus, correct Sandhi splitting is very important for subsequent NLP operations like morphological analysis, syntactic parsing, and information extraction.

Traditionally, computers have been relying mostly on rule-based systems stemming from classical grammars and dictionaries to handle Sandhi splitting. These systems, however, despite being linguistically accurate, have limited coverage, they cannot resolve ambiguities very well, and their performance on unseen or noisy data is not satisfactory. Recently, machine learning-based methods were proposed as a solution, allowing models to acquire the knowledge of Sandhi transformation patterns straight from the data.

This report is about the investigation of AI-based methods for Sanskrit Sandhi splitting. Different neural network architectures, e.g. recurrent sequence models and attention-based encoder-decoder models, were studied and coded while using a unified preprocessing scheme. Based on qualitative and quantitative evidence, the authors discuss how various paradigms work in terms of boundary recognition and phonological retrieval, also pinpointing the common mistakes they make concerning compound length and Sandhi type. The knowledge gained from the present work reveals the pros and cons of the different neural mechanisms and can be used as a springboard for the development of more reliable Sandhi splitter systems in subsequent studies.

Keywords - Sanskrit NLP, Sandhi Splitting, Phonological Restoration, Sequence-to-Sequence Models, Neural Networks, Character-Level Modeling, Attention Mechanisms, Computational Linguistics, Word Segmentation

I. INTRODUCTION

➤ Background and Motivation

Sanskrit is generally considered one of the most linguistically logical and structurally rich languages globally. It boasts a vast body of texts ranging from literature to philosophy, from science to religion, many of which are also classical Indian knowledge bases. The language features an extremely formalized grammar system, which is extensively given in Pāṇini's Aṣṭādhyāyī that sharply determines phonological, morphological, and syntactic rules. One very unique characteristic of Sanskrit phonology is Sandhi, which is a set of euphonic combination rules that regulate phonetic changes at the word or morpheme boundaries.

Sandhi changes aim at making pronunciation easier, ensuring phonetic smoothness, and conforming to metrical constraints in the case of both spoken and written Sanskrit. Nevertheless, although these changes make the language flow better, they remove obvious word boundaries from continuous text. This is why Sanskrit in its written and spoken forms frequently gives words as a single unit, thus, it is not easy to figure out the original components. This feature contrasts Sanskrit sharply with many modern languages in which word boundaries are clearly indicated by spaces or punctuation.

For computational linguistics, Sandhi introduces a major problem. It is basically an ambiguous task to find the correct places for boundaries since one surface form can be a reflection of multiple valid Sandhi splits. Apart from finding boundaries, one also has to reconstruct the original phonological forms of the component words by undoing the Sandhi changes. Thus, Sandhi splitting here is not just a segmentation problem but a hybrid task that entails both segmentation and phonological restoration. A mistake in either boundary identification or phoneme restoration might result in wrong lexical forms.

The consequences of such errors go far beyond Sandhi splitting per se. Since word segmentation serves as a primary preprocessing stage, any less-than-perfect work would affect later natural language processing (NLP) tasks such as morphological analysis, syntactic parsing, machine translation, and semantic interpretation. Even minor segmentation errors can bring about a domino effect of failures in

higher linguistic analysis levels, thus leading to a considerable overall system performance decline. Hence, having a precise and dependable Sandhi splitting procedure is essential to the construction of efficient Sanskrit NLP pipelines.

Nowadays, the increase in digitized Sanskrit materials availability has only made the issue of automated Sandhi splitting more urgent. It is totally impractical to manually annotate and correct Sandhi splits for large corpora from a time and labor perspective. On the one hand, traditional rule-based systems are very linguistically accurate; on the other hand, they are limited by their dependency on predetermined grammar rules and lexicons, thus, they cannot generalize to unknown or rare constructions. These drawbacks have stimulated the quest for data-driven methods, especially the machine learning and neural network models ones, which have the capability of identifying Sandhi patterns from annotated data as well as handling variations which are not explicitly grammatical knowledge encoded.

➤ Scope of This Work

This research work limits its scope problem of lexical segmentation of Sanskrit Sandhi using artificial intelligence and machine learning techniques. It is purely a study on classical Sanskrit only, excluding any consideration of modern adaptations, spoken variants, or downstream NLP applications. The investigation of representative neural network-based models, which have been proposed for Sandhi segmentation, and their evaluation in a controlled and consistent experimental setup are the main objectives of this work.

This report does not propose a new algorithm, or claim performance enhancement. Instead, it underlines the understanding and comparative analysis of models. The study sets out to examine the models' strategies for the two main problems: detecting of the boundary and restoring of the phonological, identifying typical failure modes, and disclosing some practical limitations during the experimentation. It demonstrates that the architectural played the role in handling ambiguity, long compound words, and restoration error.

This research concentrates on the problem of Sandhi splitting only and performs the in-depth study of it to yield the clear insights that could be helpful for the future studies of Sanskrit computational linguistics. The author's observations and analyses are meant to provide a well-organized resource and a point of reference for researchers and developers who wish to create more sophisticated, context-aware, and linguistically grounded Sanskrit NLP systems. Therefore, it is a research that makes an impact not by the introduction of novelty, but by the consolidation of understanding

and the systematic review of the existing AI-based Sandhi splitting approaches.

II. LINGUISTIC BACKGROUND : SANSKRIT SANDHI

➤ Overview of Sandhi

Sandhi represents in Sanskrit a set of closely related phonological rules that explain the changes in sound when morphological units or words come very close. The word Sandhi literally means "joining" or "connecting," and it chiefly functions to make spoken language easier and smoother by physically and phonetically more comfortable transitions. Besides, these changes in phonetics also help in maintaining the rhythmic and metrical patterns of Sanskrit, especially in the case of poetry and chant recitals. The original lexical units get so drastically changed by the phonological consequences of Sandhi that any inquiry into their underlying structure becomes difficult.

Once a pair of words has been produced, then what is heard as the last phoneme of the first word and the first phoneme of the next one is sometimes merged, altered, or the whole sound is eliminated. Such changes in the phonetic system are not without reason. They originate from beautifully structured grammatical rules set out in classical Sanskrit grammar. In this way, the recited Sanskrit language gets a very pleasing and harmonious sound whereas its written form at the same time lacks clear word separators. Instead of using spaces to separate words as many modern languages do, Sanskrit usually gives the impression of continuously running character sequences enjoyed in the phrase or sentence.

Looking at it from the computational angle, Sandhi causes great difficulty because the structure of the sentence and the boundaries of the words are almost invisible. One uninterrupted string of characters may stand for several legitimate word sequences and each of them must be correctly resegmented accurately to original phonological forms. Without extra syntactic or semantic clues, it is almost impossible to decide on the proper division of the text. The consequence of this is that the task of Sandhi splitting that is fundamental to Sanskrit Natural language processing requires linguistic knowledge as well as computational power and resilience.

➤ Major Types of Sandhi

The sound change phenomena of Sanskrit Sandhi can generally be divided based on whether vowels or consonants are the transforming parts. Even though classical grammar mentions numerous minor types, only

the main types from a computational perspective are dealt with in this article.

- Vowel Sandhi

Vowel Sandhi is present when a word ending in a vowel is immediately succeeded by a word starting with a vowel. The two vowels can be replaced by just one vowel or a third different vowel in such cases. The changes usually involve the length or quality of the vowel.

The major hurdle for Vowel Sandhi for computational models is that a number of vowel pairs are indistinguishable from a single vowel sound. To reverse the process one has to learn the delicate phonological rules and be able to identify the vowels that were taken apart.

- Consonant Sandhi

Consonant Sandhi is the collective term for the changes made to the consonants at the end of one word and the beginning of another. Depending on the situation, the alterations could be assimilation, voicing, devoicing, and even the addition of nasal sounds.

From the standpoint of splitting Sandhi, the consonant Sandhi makes it hard to locate the boundaries as the original consonants might have been lost in the final text. Therefore, neural models are required to predict the occurrence of the hidden boundaries and they reconstruct the suitable consonants through the patterns they have learned.

- Visarga and Anusvāra Sandhi

Phonetic elements like visarga (◌:) and anusvāra (◌̣) change their forms based on the circumstances in Sandhi. The most common changes are ones that are not actually denoted in the surface script, thereby heightening the ambiguity.

Computationally, there is no good way to deal with visarga and anusvāra Sandhi if the sound after the transformation is not known because this would require the model to recognize the local phonetic context.

➤ Computational Challenges in Sandhi Splitting

Despite the fact that Sandhi rules are linguistically very systematic, their computational reversal is quite difficult. Firstly, Sandhi splitting not only needs to find the right spots for splits but also has to restore the original phonological forms of the words. Mistakes at either step will give a wrong result.

Secondly, Sandhi splitting is ambiguous by nature. One surface form can especially denote more than one valid splits if there is no sentence-level or semantic context to explain it. In such cases, a system based solely on the rules hardly knows which of the several possible meanings is the most likely, whereas a model that is

data-driven has to guess from the regularities of the statistics it has learned from the training data.

Third, Sanskrit compounds can be very long or very short. It is usually easy to split very short compounds, but longer ones are more difficult since they involve multiple Sandhi changes and the problem of splitting correctly is even more complicated. It has been observed that when the length of a compound goes up, the performance of neural models goes down, which means that recognizing the importance of long-range dependencies leads to better results.

To conclude, there could be more noise due to changes in script representation, orthography, and data quality. Differences between Devanagari and transliterated representations, irregular use of diacritics, and mistakes in digitized manuscripts are all things that have negative consequences on the performance of a model. Thus, well-organized preprocessing and character-level modeling are the right ways to make Sandhi splitting systems work well.

➤ Relevance to This Work

Knowledge of the linguistic features of Sandhi is necessary for creating and assessing computational models for Sandhi splitting. The models discussed in this report mainly focus on the character level and learn the transformation patterns directly from the data rather than relying on explicitly encoded grammatical rules. For that reason, it makes sense to view their strengths and weaknesses in light of the linguistic phenomena presented in this section.

This work, through the incorporation of the linguistics aspect, brings forth why neural models can be great with some of the Sandhi cases while the rest keep being difficult. This angle assists not only in the current results' assessment but also in the formulation of future improvement plans.

III. LITERATURE REVIEW

Research on Sanskrit Sandhi splitting underwent a dramatic transformation over time, gradually moving from linguistically motivated, rule-based systems to data-driven machine learning and neural approaches. This transition mirrors the overall development of natural language processing, where linguistic rules handcrafted by humans are increasingly supplemented or replaced by statistical and learning-based models. In the case of Sanskrit, the transition holds a particular importance owing to the language's complex morphology and highly systematic phonological structure. The present chapter discusses the main computational techniques used for Sanskrit Sandhi splitting and will focus more on neural and AI-based

methods, the latter being most relevant to the present work.

A. Rule-Based and Lexicon-Driven Approaches

Initially, computational methods for Sanskrit Sandhi splitting were deeply intertwined with Pāṇini's formal grammatical framework. Among many others, the most influential and widely cited in this series is undoubtedly the Sanskrit Heritage Engine created by Huet [1]. The system represents Pāṇinian phonological and morphological rules using finite-state techniques and unites them with a large-scale lexicon. When provided with a literal string, the system basically lists all the grammatically correct Sandhi splits, hence, it is completely consistent with the grammatical rules of classical Sanskrit. Consequently, the system's output is linguistically accurate, and every produced split may be directly linked to a particular grammatical rule in the language.

Taking further steps based on this main work, Goyal and Huet [2] performed a very detailed study on the completeness of grammar-driven analyzers with the result that a well-equipped rule-based system supplemented with a very comprehensive lexicon can theoretically generate all valid Sanskrit compound segmentations. This finding not only confirmed the linguistic correctness of the Pāṇini grammar-based methods but also pointed to their significance as the essential instruments of Sanskrit computational linguistics.

Kulkarni et al. [3] presented Samsādhānī, an all-inclusive Sanskrit computational toolkit which makes Sandhi splitting a part of itself, alongside with morphological analysis and syntactic parsing. Their methodology is mostly dependent on the use of manually authored grammatical rules and lexicons derived from the ancient, traditional Sanskrit texts. In a similar vein, Mittal [4] carried out work on lexicon-driven Sandhi splitting and ended up with a finite-state transducer implemented on the basis of phonological rewrite rules and a dictionary lookup

only. These methods are extremely strong, interpreting-wise, where the transparency of each split's rationale is a feature deeply rooted in the well-established grammatical theory.

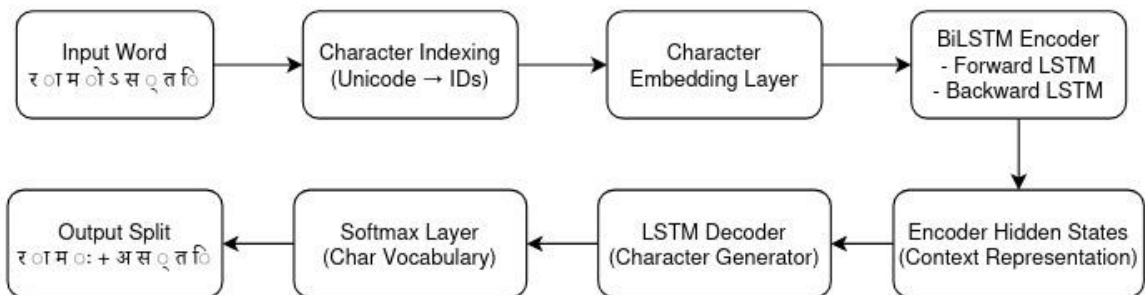
While rule-based systems are perfectly grounded in linguistics, their shortcomings have been, to some extent, an accepted fact in the scientific community. Bhardwaj et al. [5] noted that, as a result, such systems have a limited vocabulary when they come across rare words, newly occurring words or compounds that are not present in the lexicon they are based on. Besides that, the thoroughness of the grammatical rules that have been applied can lead to the generation of a large number of grammatically and even semantically valid splits without a clear way to choose the most reasonable one if the context or the meaning is not known. Therefore, while rule-based methods continue to be appreciated from a linguistic point of view, they still have the problem of scalability and ambiguity resolution when it comes to dealing with large, real-world corpora.

B. Machine Learning and Sequence Labeling Approaches

To overcome the scalability restrictions of rule-based systems, the community of researchers started to represent Sanskrit Sandhi splitting as a supervised machine learning problem. One of the foremost endeavors in this path was undertaken by Hellwig [6], who approached Sanskrit compound splitting as a character-level sequence labeling problem and utilized recurrent neural networks. Under this setup, a label is given to each character of the input sequence identifying the occurrence of a Sandhi boundary at that place. The method allows the model to extract phonological patterns from the data implicitly without the need for explicitly encoded grammatical rules.

Sequence labeling methods showed better generalization and scalability than rule-based systems, especially on the application of large corpora. However,

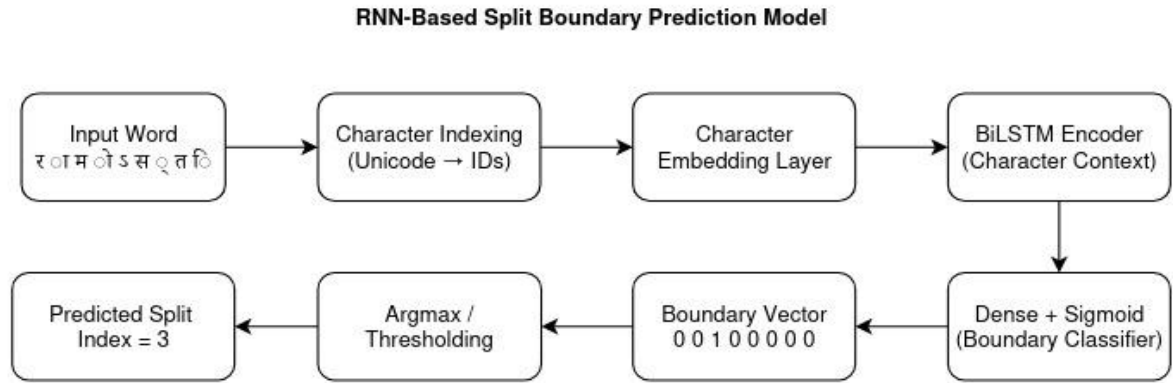
Seq2Seq Character-Level Sandhi Splitting Model



few studies pointed out a significant drawback of this approach. Reddy et al. [9] stated that sequence labeling models are mainly concerned with boundary detection and thus, do not deal with the problem of phonological restoration in a direct manner. After the detection of the boundary positions, the original lexical forms that got changed by the Sandhi transformations need to be restored through the subsequent reprocessing steps.

decoder to align the input text character sequences with the output segments that it was to generate, thus leading to higher accuracy. These models are free from any intermediate step, such as boundary label prediction, as they directly output unsandhified strings. This explains why their performance is superior to that of sequence labeling.

However, some drawbacks have been spotted. Seq2Seq models may not perform very well especially



Performed error analyses point out that errors in restoring deleted or altered phonemes make up a major portion of the overall failures of such systems. Hence, despite sequence labeling being superior to conventional rule-based systems with respect to flexibility and coverage, it is still dependent on the existence of handcrafted post-processing pipelines for its operation. This dependence restricts its capability to be a complete end-to-end solution for Sandhi splitting.

C. Sequence-to-Sequence Models for Sandhi Splitting

Sandhi splitting was traditionally a complex task involving a lot of linguistic rules but it has recently been treated as a string-to-string transformation task by sequence-to-sequence (Seq2Seq) models. This result in a paradigmshift in this matter. Aralikatte et al. [10] proposed a novel double-decoder RNN architecture where one decoder outputs the Sandhi split positions and the other reconstructs the constituent words. Their joint boundary identification and phonological restoration model eliminates the need for explicit post-processing rules. The experiments they ran showed that their technique is significantly better than tagging-based methods that were the baseline.

Reddy et al. [9] in their work demonstrated that even standard encoder-decoder models with attention mechanisms could be effectively trained for Sanskrit word segmentation with only a small amount of data. The addition of an attention mechanism allowed the

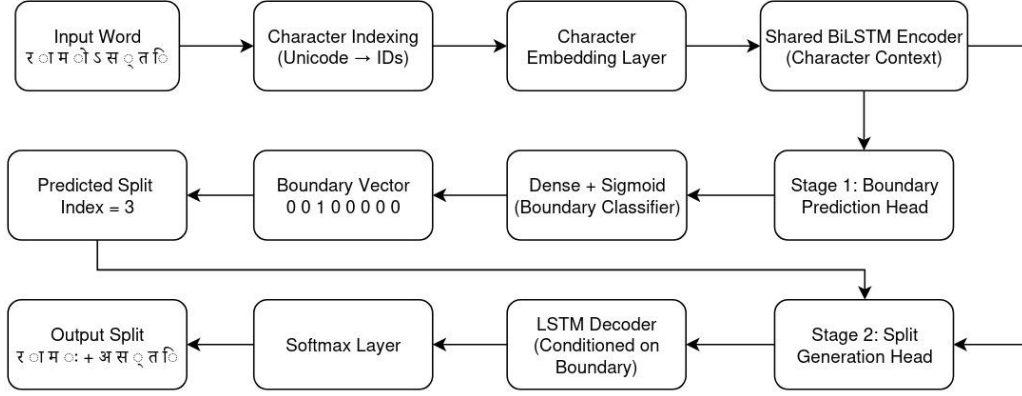
on extremely long compounds that have a great number of morphemes due to the large sequence length which is a factor of the issue. It is also claimed that the accuracy of the model is highly correlated with the increase in size and diversity of the training data, thus low-resource settings may struggle to achieve good performances [10]. For all their drawbacks, Seq2Seq models still represent a significant step towards end-to-end Sandhi splitting.

D. Two-Stage Neural Architectures

One way to address the difficulty of direct Seq2Seq learning is by breaking down Sandhi splitting into multiple stages. A two-stage neural framework that localizes the Sandhi window in the first stage and the second model secondly splits and restores within the window was presented by Dave et al. [11]. This led to sequence length reduction and learning excessively being focused on the area affected by the phonological changes.

Hellwig and Nehrdich [12] mixed boundary prediction using a neural network with grammatical constraints derived from Paninian rules for the purpose of ruling out invalid splits. Hybrid setups of this kind try to keep the linguistic interpretability intact whilst statistical learning is used for disambiguation purposes.

Two-Stage Neural Model for Sanskrit Sandhi Splitting



On the other hand, using multiple stages in an architecture can cause errors to propagate from one stage to the next. Incorrect localization at the first stage as given in the study by Dave et al. [11] means that the next stage would turn up to be faulty too. In a nutshell, two-stage systems do offer benefits in interpretability as well as modularity but their overall robustness is greatly dependent on the early stages' accuracy.

E. Transformer-Based and Character-Level Models

Recent studies have explored Transformer architectures for Sanskrit Sandhi splitting due to their ability to model long-range dependencies. Bhatt et al. [13] proposed CharSS, a character-level Transformer model that avoids tokenization entirely. Their results demonstrated significant performance gains over recurrent and Seq2Seq models, particularly for long compounds.

Sandhan et al. [14] investigated linguistically informed Transformer tokenizers for Sanskrit, showing that self-attention mechanisms capture complex phonological interactions effectively. Byte-level Transformer models have also been evaluated, as they naturally handle unseen characters and avoid vocabulary limitations.

Despite their effectiveness, Transformer-based models are computationally expensive and require substantial training data. Bhatt et al. [13] noted that character-level Transformers significantly increase memory and training costs. As a result, their adoption is often constrained by resource availability.

F. Observations and Research Gaps

Upon a thorough analysis of the literature concerning Sanskrit Sandhi splitting, a number of recurring themes

and general tendencies stand out. Most significantly, a clear and well-charted shift can be seen from typical, rule-based linguistic systems to more modern, data-driven and neural network-based models. This transition is, to a large extent, a reflection of the general trend in natural language processing research, where explicitly designed grammatical rules are gradually being supplemented or altogether replaced by learning-based techniques capable of extracting intricate patterns from data. In Sanskrit's case, this transformation has been necessitated by the desire to manage extensive corpora, linguistic variations, and the ambiguities that the introduction of Sandhi transformations brings about. Besides that, a great deal of literature often emphasizes character-level modeling as an indispensable, key component of the model's architecture that can be instrumental in solving Sandhi problems since phonological alterations are indeed made at the character and sound level.

However, there are still quite a few avenues that have not been sufficiently scrutinized. One of the major shortcomings across the board has been the inconsistency of the experimental evaluation. The majority of papers review their models based on different datasets, preprocessing pipelines, annotation standards, and evaluation metrics, which, in turn, makes comparing the different methods directly quite challenging and, in some cases, even unreliable.

Besides that, while numerical performance indicators are almost always presented, comprehensive and in-depth qualitative error analysis is frequently scarce or totally lacking. There is hardly any conversation, if at all, on the real linguistic or structural factors as to why the Sandhi splits were incorrect, e.g., whether the mistakes are actually the result of boundary error, phonological restoration, or even ambiguity of underlying forms. Such limitations point towards the necessity of the systematic re-examination of the tasks under stable and well-defined experimental settings that

ensure the comparability of results and the deeper understanding of the behavior of models.

Based on these considerations, this communication extends the corpus of knowledge by utilizing neural network-based techniques for the Sandhi splitting task in Sanskrit. The work is not geared towards the development of new models but rather, through the careful execution, regulated evaluation, and qualitative scrutiny, aims to reveal the underlying strengths and weaknesses of previously existing models. The experimental approach used here may help provide more clarity in understanding model behavior and practical difficulties involved in Sandhi splitting thus paving the way for subsequent research in Sanskrit computational linguistics.

IV. PROBLEM STATEMENT

➤ Task Description

Sanskrit Sandhi splitting is a complex linguistic task that aims to take a sandhied or compound word at the surface level and break it down into the original individual words while also changing their phonological forms to the canonical ones. Simply put, the problem's input is a run of characters with no explicit word boundaries because the words have been joined together according to the Sandhi rules. So the goal is twofold: to find the exact spots where the string can be divided into segments and to get the original words back as they were before the Sandhi transformations.

More precisely, the task is to produce the output word sequence

$$W = \{w_1, w_2, \dots, w_n\}$$

that the concatenated surface form S originated from after the application of the appropriate Sandhi rules. In terms of computation, the result is most often a string consisting of the constituent words separated by some specific delimiter, i.e., a '+' sign, which clearly indicates word boundaries and separates the restored forms from the sandhied input.

In languages where word boundaries are made clear by the usage of white spaces or punctuation marks, the process of segmentation is usually a straightforward preliminary step. However, Sanskrit is a language that offers a much bigger challenge. The changes involved in Sandhi may be quite intricate and complicated, affecting vowels, consonants, or even special phonetic markers like visarga (◌:) and anusvāra (◌ṃ). Such alterations may include the disappearance, the substitution, or the assimilation of sounds at the borders of words, thus making it difficult to determine both the exact location of the boundaries and the canonical forms of the words.

As a result, the problem cannot be limited to just finding the boundaries.

In brief, Sanskrit Sandhi splitting is essentially a dual problem. It is about finding the word boundaries in a continuous string of characters and at the same time being able to reconstruct the phonological forms of the words accurately. Hence, any computer-based method for this matter must take into consideration both the division and the phonological restoration so as to generate output that is not only linguistically valid but also meaningful.

➤ Challenges Addressed

Sandhi splitting faces a variety of unique challenges that set it apart from the typical sequence segmentation tasks in other languages. Firstly, the difficulty of defining boundaries is a core problem in sandhi splitting. The phenomenon of sandhi does not leave behind clear indicators or signals that show where word boundaries are. On the other hand, phonological modifications made at the time of Sandhi might change or merge the sounds at the word boundaries so that the resulting form is hardly recognizable as a concatenation of words. In this context, one letter string can be divided into several correct segments at the linguistic level, which is especially true for those that are due to common and regular Sandhi transformations.

Furthermore, apart from boundary detection, Sandhi breaking is also about phonological restoration. The job does not end with identifying the right position of the split; the original forms of the words must be reconstructed as well. This process may include adding back what was removed during the application of Sandhi, changing the length or quality of the vowel, or fixing the consonants that have been affected by such processes as assimilation, voicing, etc. Hence, it is not possible to regard Sandhi breaking as a mere structural segmentation task only, but at the same time, phonological reconstruction in a linguistically coherent way should be involved.

Moreover, ambiguity can be described as a factor that makes the task even more difficult. A large number of sandhied forms can have more than one grammatically correct split, thus resulting in different constituent words that may also have different meanings. When a model has no access to sentence-level or semantic context, it just has to trust learned patterns, inherent probabilities, or statistical properties in the training data to choose the most likely segmentation. Such ambiguity by nature hinders making a decision with certainty and also raises the chance of mistakes, which is especially true for infrequent or context-sensitive cases.

What adds to the complexity, to a large extent, is the variable length of the compounds. Short compounds might only have been subject to a single Sandhi change

and are generally simpler in their analysis. By way of contrast, longer compounds can be said to have undergone multiple Sandhi changes, one after another, which is equivalent to a significant increase in the complexity of both boundary detection and phonological restoration. At the same time, the longer the compound gets, the more the ability to take long-range dependencies of the character sequence into account matters. Those models that lack the necessary features to be able to hold on to the contextual information for quite long ways typically show lower performance on such inputs, thus pointing out the necessity of the architectures that can handle extended dependencies.

➤ Scope and Assumptions

The scope of the present work is deliberately restricted to **word-level Sanskrit Sandhi splitting**. No consideration is given to sentence-level context, syntactic structure, or semantic disambiguation. Each input instance is assumed to be a single sandhied word or compound presented in a normalized script representation. By isolating the task at the word level, the study avoids the additional complexity introduced by contextual reasoning and focuses exclusively on the core linguistic and computational challenges inherent to Sandhi splitting.

To ensure methodological consistency and enable fair comparison across different models, several simplifying assumptions are adopted. First, Sandhi is treated purely as a phonological phenomenon, without accounting for semantic or pragmatic influences that may affect interpretation at the sentence level. Second, it is assumed that script-level variations introduced by Sandhi transformations are minimal, thereby reducing noise caused by orthographic inconsistencies or alternative representations. This assumption allows the models to concentrate on phonological reconstruction rather than script normalization.

Additionally, the models are permitted to output sequences of morphemes rather than being constrained to raw text strings alone. This design choice reflects the linguistic reality of Sanskrit, where meaningful segmentation often aligns more naturally with morphemic units than with surface word forms. Allowing morpheme-level outputs provides flexibility in representing restored forms while maintaining linguistic validity.

Under these assumptions, Sanskrit Sandhi splitting can be formulated as a controlled sequence modeling problem. This formulation makes it possible to analyze and compare different neural architectures within a stable experimental framework, without confounding effects from higher-level linguistic context. As a result, the study emphasizes architectural behavior, modeling capacity, and practical limitations, rather than contextual inference or semantic resolution.

➤ Objective of This Work

With the problem formulation clearly established, the primary objective of this work is to examine how different neural modeling approaches handle the task of Sanskrit Sandhi splitting within a unified and controlled experimental setup. The focus is not limited to whether a model can successfully generate a segmentation, but also extends to understanding the extent to which it internalizes and represents Sandhi-related phonological transformations. In particular, attention is given to how effectively models capture both boundary information and the underlying phonological patterns required for accurate reconstruction.

By explicitly defining the scope and limitations of the task, this section provides a structured foundation for the subsequent stages of the study. It sets the context for dataset preparation, model selection, and experimental evaluation, all of which are discussed in the following sections. This clear delineation ensures that the analysis remains focused on the intended objectives and allows the results to be interpreted within well-defined constraints.

V. DATASET AND PREPROCESSING

➤ Dataset Description

The experiments mentioned here are based on an annotated Sanskrit Sandhi dataset made up of compound words (sandhied) along with their constituent parts (split forms). To put it simply, the examples from this data embody a surface-level compound, the result of the Sandhi rules being applied, and the matching breakdown of the original simpler words. These decomposed word forms are the gold-standard annotations, thereby providing clear evidence of word boundaries and phonological restoration.

Such datasets are mainly sourced from well-known Sanskrit linguistic resources and corpora that have been effectively utilized in the previous research work on Sandhi splitting. The dataset is at the single-word level rather than the sentence level, which is why it is very good for the isolated study of Sandhi without the influence of the syntactic or semantic contexts. Each record not only indicates the points of segmentation but also captures the phonological changes that have been introduced through the Sandhi application, thereby offering a complete picture of the transformation process.

The dataset structure is quite ideal for character-level machine learning, as both the input (sandhied form) and the output (split form) are sequences of characters. This also makes more sense given the phonological nature of Sandhi because most of the time the transformations take place at the level of individual characters or graphemes, not at the level of the whole word. Therefore, character-based neural networks can pick up phonological patterns directly from the example data, thus making it unnecessary to instruct them explicitly through rules.

चरकसंहिता
सूत्रस्थानम्।
--*--
प्रथमोऽध्यायः।

<1-1>
अथातो दीर्घज्जीवितीयमध्यायं व्याख्यास्यामः॥ 1 ॥
इति ह स्माह भगवानात्रेयः॥ 2 ॥

(आयुर्वेददीपिका)

गुणत्रयविभेदेन मूर्तित्रयमुपेयुषे।
त्रयीभूवे त्रिनेत्राय त्रिलोकीपतये नमः॥ 1 ॥
सरस्वत्ये नमो यस्याः प्रसादात् पुण्यकर्मभिः।
बुद्धिदर्पणसंकान्तं जगदध्यक्षमीक्ष्यते॥ 2 ॥
ब्रह्मदक्षाश्चिदेवेशभरद्वाजपुनर्वसु-
हुताशवेशचरकप्रभृतिभ्यो नमो नमः॥ 3 ॥
पातञ्जलमहाभाष्यचरकप्रतिसंस्कृतेः।
मनोवाक्कायदोषाणां हर्त्रेऽहिपतये नमः॥ 4 ॥
नरदत्तगुरुर्द्विष्टचरकार्थानुगामिनी।
क्रियते चक्रदत्तेन टीकाऽऽयुर्वेददीपिका॥ 5 ॥
सभ्याः सदगुरुवाक्सुखसुतिपरिस्फूर्तश्रुतीनस्मि वो
नालं तोषयितुं पयोदपयसा नाम्भोनिधिस्तुष्यति।
व्याख्याभासरसप्रकाशनमिदं त्वस्मिन् यदि प्राप्यते

Figure 4: Original Text

One of the main sources of quality annotations featured in this paper is the choreographed manual Sandhi-split corpus of the Charaka Samhitā, developed by the University of Hyderabad [15]. This work covers the entire three decades of chapters, providing sandhied forms as well as split correspondence from expert annotators. The corpus not only has manually verified annotations but is also based on classical Sanskrit texts which make it an authoritative source of gold-standard data and has become a common tool for Sanskrit Sandhi splitting research.

Manually curated editions are of exceptional worth in both the training and testing of neural models, as they essentially eradicate the noise in the annotations while still keeping authentic phonological transformations intact.

Every single instance of the data has the same backbone, i.e., a sandhied word serving as input and a split representation output that matches the input. This setup essentially turns the Sandhi splitting challenge into a problem of supervised machine learning, where models learn to produce segmentations or splittings given the input sequence of characters.

Both input and output are character-level representations, which means that the model is induced to infer the phonological rules from raw data rather than

चरकसंहिता सूत्रस्थानम्

...*

प्रथमः अध्यायः

(प्रो.ओ.इ. चोम्प्लेटेड) <1-1>
अथातो दीर्घ-ज्जीवितीयम्+अध्यायं व्याख्यास्यामः॥ 1 ॥

गुणत्रयविभेदेन मूर्तित्रयम्+उपेयुषे
त्रयीभूवे त्रिनेत्रायत्रिलोकीपतये नमः॥1॥
सरस्वत्ये नमः+ यस्याः प्रसादात्पुण्यकर्मभिः
बुद्धिदर्पणसंकान्तं जगत्+अध्यक्षम्+ईक्ष्यते॥2॥
ब्रह्मदक्ष+अक्षिदेवेशभरद्वाजपुनर्वसुहुताशवेशचरकप्रभृतिभ्यः+नमः(1) (1. भिक्तः+नमः इतिपाठः) नमः॥3॥
पातञ्जलमहाभाष्यचरकप्रतिसंस्कृतेः
मनोवाक्कायदोषाणां हर्त्रे+अहिपतये(2) (2. हुन्ते
इति
) नमः॥ 4 ॥
नरदत्तगुरु+उद्दिष्टचरक+अर्थ+अनुगामिनी
क्रियते चक्रदत्तेन टीका+आयुर्वेददीपिका॥5॥
सभ्याःसदगुरुवाक्सुखसुतिपरिस्फूर्तश्रुतीनस्मि वः+
न+अतं तोषयितुं पयोदपयसा न+अम्भोनिधिः+तुष्यति

Figure 5: Manually Annotated Text

garnering explicit rules from linguists or prior knowledge. This approach is a plus for neural networks as they get a chance to learn phonological patterns associated with Sandhi transformations that they implicitly encode. Additionally, the explicit notation of word boundaries in the output makes it easier to carry out evaluation and also comparison across various models.

```
{
  "original": "अङ्गिरा जमदग्निश्च वसिष्ठः कश्यपो भृगुः। आत्रेयो गौतमः साङ्ख्यः पुलस्त्यो  
नारदोऽसितः",
  "sandhi_split": "अङ्गिरा जमदग्निः+च वसिष्ठः कश्यपः+ भृगुः आत्रेयः+ गौतमः साङ्ख्यः पुलस्त्यः+  
नारदः+असितः"
},
{
  "original": "अगस्त्यो वामदेवश्च मार्कण्डेयाश्चलायनौ। पारिक्षिर्भिक्षुरात्रेयो भरद्वाजः कपिञ्ज(छ)लः",
  "sandhi_split": "अगस्त्यः+ वामदेवः+च मार्कण्डेयः+अश्चलायनौ पारिक्षिः+भिक्षुः+आत्रेयः+ भरद्वाजः  
कपिञ्ज(छ)लः"
},
{
  "original": "विश्वामित्राश्वमेधयो च भार्गवश्च्यवनोऽभिजित्। गार्ग्यः  
शाण्डिल्यकौण्डिल्यौ(न्यौ)वार्क्षिर्देवलगालवौ",
  "sandhi_split": "विश्वामित्रः+आश्वमेधयो च भार्गवः+च्यवनः+अभिजित्+गार्ग्यः+शाण्डिल्यकौण्डिल्यौ(न्यौ)वार्क्षिर्देवलगालवौ"
},
{
  "original": "साङ्कल्यो वैजवापिश्च कुशिको बादरायणः। बडिषः शरलोमा च काप्यकात्यायनावुभौ",
  "sandhi_split": "साङ्कल्यः+ वैजवापिः+चकुशिकः+ बादरायणः+बडिषः शरलोमा च काप्यकात्यायनौ+उभौ"
},
{
  "original": "काङ्कयनः कैकश्यो धौम्यो मारीचकाश्वपौ। शर्कराक्षो हिरण्पाक्षो लोकाक्षः पैङ्गिरेव च",
  "sandhi_split": "काङ्कयनः कैकश्यः+ धौम्यः+ मारीचकाश्वपौ+शर्कराक्षः+ हिरण्पाक्षः+लोकाक्षः  
पैङ्गिः+एव च"
},
{
  "original": "अङ्गिरा जमदग्निश्च वसिष्ठः कश्यपो भृगुः। आत्रेयो गौतमः साङ्ख्यः पुलस्त्यो  
नारदोऽसितः",
  "sandhi_split": "अङ्गिरा जमदग्निः+च वसिष्ठः कश्यपः+ भृगुः आत्रेयः+ गौतमः साङ्ख्यः पुलस्त्यः+  
नारदः+असितः"
},
{
  "original": "अगस्त्यो वामदेवश्च मार्कण्डेयाश्चलायनौ। पारिक्षिर्भिक्षुरात्रेयो भरद्वाजः कपिञ्ज(छ)लः",
  "sandhi_split": "अगस्त्यः+ वामदेवः+च मार्कण्डेयः+अश्चलायनौ पारिक्षिः+भिक्षुः+आत्रेयः+ भरद्वाजः  
कपिञ्ज(छ)लः"
},
{
  "original": "विश्वामित्राश्वमेधयो च भार्गवश्च्यवनोऽभिजित्। गार्ग्यः  
शाण्डिल्यकौण्डिल्यौ(न्यौ)वार्क्षिर्देवलगालवौ",
  "sandhi_split": "विश्वामित्रः+आश्वमेधयो च भार्गवः+च्यवनः+अभिजित्+गार्ग्यः+शाण्डिल्यकौण्डिल्यौ(न्यौ)वार्क्षिर्देवलगालवौ"
},
{
  "original": "साङ्कल्यो वैजवापिश्च कुशिको बादरायणः। बडिषः शरलोमा च काप्यकात्यायनावुभौ",
  "sandhi_split": "साङ्कल्यः+ वैजवापिः+चकुशिकः+ बादरायणः+बडिषः शरलोमा च काप्यकात्यायनौ+उभौ"
},
{
  "original": "काङ्कयनः कैकश्यो धौम्यो मारीचकाश्वपौ। शर्कराक्षो हिरण्पाक्षो लोकाक्षः पैङ्गिरेव च",
  "sandhi_split": "काङ्कयनः कैकश्यः+ धौम्यः+ मारीचकाश्वपौ+शर्कराक्षः+ हिरण्पाक्षः+लोकाक्षः  
पैङ्गिः+एव च"
},
{
  "original": "अङ्गिरा जमदग्निश्च वसिष्ठः कश्यपो भृगुः। आत्रेयो गौतमः साङ्ख्यः पुलस्त्यो  
नारदोऽसितः",
  "sandhi_split": "अङ्गिरा जमदग्निः+च वसिष्ठः कश्यपः+ भृगुः आत्रेयः+ गौतमः साङ्ख्यः पुलस्त्यः+  
नारदः+असितः"
}
```

Figure 6: Parallel Corpus made using the manually annotated text

➤ Preprocessing Pipeline

To keep things fair when comparing different models, the same preprocessing steps are applied to the dataset regardless of the model.

Initially, all the input texts are converted to a single representation of the script. The rationale behind this is that the Sanskrit language can be written in various scripts or can be transliterated differently. Normalizing the data means that the models are not misled by

changes in spelling and can concentrate on learning the language's phonetic changes due to Sandhi.

Moreover, words and sentences considered as errors or of low quality are deleted. They can be incomplete splits, wrong labels, or even non-Sanskrit characters. By cleaning the data, the model gets less confused and learns less of the undesired patterns.

After normalization and cleaning operations, the texts of both questions and answers are changed into a series of characters one by one. This character-level means of expressing the texts is universally applied among the models experimented with in this study. The use of special characters, such as the separator showing the edges of words, is allowed in the output sentence to overtly give the segmenting information.

➤ Data Preparation for Model Training

Once the dataset has been preprocessed, it is split into different sets for training and testing purposes. Absolutely no sandhied pairs are allowed to be in the different sets simultaneously to prevent data leakage. This split allows a more reliable evaluation of a model's capability to deal with new Sandhi forms.

Next, the data is saved in different structures that are compatible with various neural architectures. Character sequences and corresponding boundary labels form the training pairs for sequence labeling models, whereas sequence-to-sequence models' training pairs consist of input character sequences and output character sequences with explicit word separators. Underneath these differences, all models rely on the same cleaned and normalized dataset, thus providing an impartial platform for comparison.

➤ Relevance to the Present Work

The dataset and preprocessing choices described above are designed to support a focused investigation of Sanskrit Sandhi splitting using neural models. By maintaining a consistent data representation and preprocessing pipeline, the effects of architectural differences between models can be analyzed more clearly. This approach also ensures that observations made in later sections are attributable to model behavior rather than inconsistencies in data preparation.

VI. METHODOLOGY

This report utilizes a two-step neural network design for the process of Sanskrit Sandhi splitting. This approach is based on the idea that segmenting the boundaries and figuring out the original sounds of the language are two very different problems. The proposed method doesn't try to learn both functions at the same

time. It distinctly fixes the task of finding the boundaries of sandhi first and then performing the lexical restoration. Thus, the method gains in terms of results and being able to explain the model. The main idea of the used approach goes back to the sandhi-window notion of Dave et al.. However, it has been adapted and extended by the way of practical implementation and empirical evaluation.

➤ Stage-1: Sandhi Boundary Detection

The first stage works on sandhi splitting at a character level, basically, trying to find the boundaries. To comprehend the context from both sides of every character, a Bidirectional Long Short-Term Memory (BiLSTM) network is used. The output of the model is a probability score for each character position, which tells how likely it is that a sandhi boundary is found at that index.

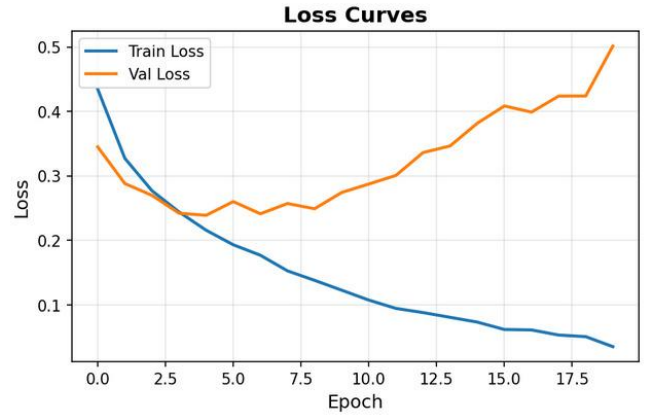


Figure 7: Training and validation loss curves across epochs for RNN Model

Word embeddings of characters are used to represent the input word and the embeddings are trainable and passed through the BiLSTM. Afterwards, a linear projection layer maps the contextual representations to a single number of boundary logits. At the time of testing, the boundary position is decided either by maximum probability or by a probability threshold set via the F1-score analysis thorough a tune, which is the validation.

The metrics used in this stage are precision, recall, and F1-score. Boundary detection is a binary classification task at the character-level and these metrics are used to evaluate it. The use of F1-score helps to be less sensitive to the problem of class imbalance between boundary characters and non-boundary characters that is inherent.

➤ Stage-2: Lexical Restoration with Sequence-to-Sequence Modeling

Once the stage of sandhi boundary detection is nailed down, the focus of the second stage is on the reconstruction of the original lexical forms. The task at hand is a character-level sequence-to-sequence problem. To generate the restored split form, an encoder-decoder architecture with attention is utilized.

The encoder gets the bidirectional LSTM to encode the sandhi window, yielding contextual features capturing phonological signals from both directions of the boundary. The decoder autoregressively generates the output sequence, at every decoding step attending to the relevant encoder states. To direct the generation, special start-of-sequence (SOS) and end-of-sequence (EOS) tokens are introduced during the training.

For the right decoding during inference, the decoding stops at the EOS token and post-processing is done to strip any remaining control tokens. The post-decoding clean-up is crucial to get the sandhi splits in a nice well-formed state that can be used for evaluation and deployment.

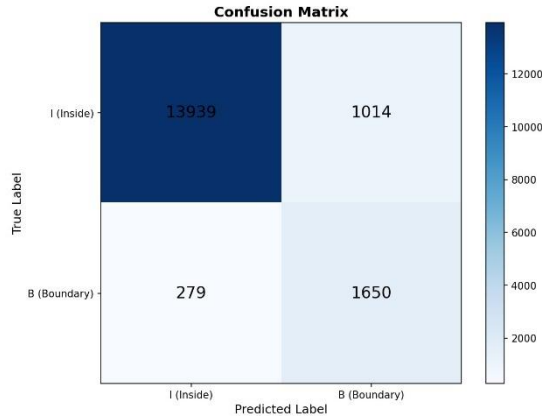


Figure 8: Confusion matrix for character-level split boundary classification

➤ Training Strategy

The training of both stages is carried out separately to avoid the propagation of errors during the learning process. Binary cross-entropy loss is the objective for the boundary detection model, whereas the restoration model uses categorical cross-entropy loss for character predictions. During Stage-2 training, teacher forcing is utilized for which there is evidence of better convergence stability.

The code for all models is written in PyTorch. They work at the level of characters and do not take pretrained language models as a starting point. The decision goes in favor of model interpretability and also brings to light the intrinsic challenge of Sanskrit sandhi restoration under low-resource scenarios.

➤ Inference Pipeline

The longest possible sandhi boundary of the input sandhied word is detected by the boundary detection model. Then the boundary thus predicted is utilized in the preparation of the sandhi window which is given to the restoration model for the production of the split.

To facilitate everyday practical use, the trained models are serialized and they have a command-line inference interface for deployment. The interface supports input files in CSV or TSV format and ensures reproducibility and ease of evaluation by generating structured outputs that contain splits and boundary indices predicted.

➤ Evaluation Metrics

First, the performance of Stage-1 undergoes a detailed evaluation by considering precision, recall, and F1-score, along with a study of the confusion matrix in order to analyze false positives and false negatives. The performance of Stage-2 along with the end-to-end one is measured by Exact Match Accuracy and Character Error Rate (CER).

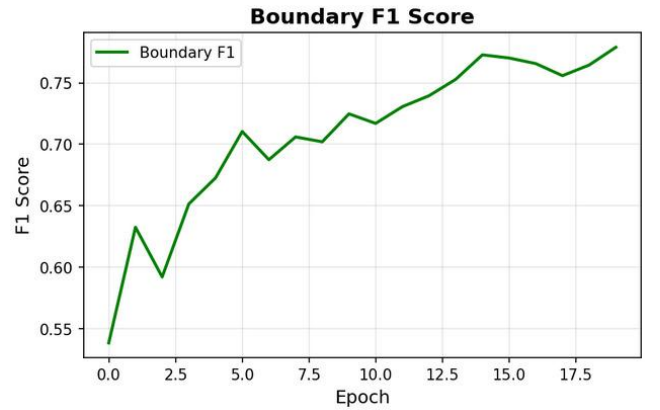


Figure 9: Boundary-level F1 score progression during RNN model training

To take into account that the sound is partly the same even if there are no exact matches, the normalized edit distance is used for the calculation of CER.

Considering the very time-consuming nature of autoregressive decoding and the large-scale dataset, end-to-end evaluation is performed on a dataset portion randomly drawn which equals the conventional evaluation of neural sequence modeling literature.

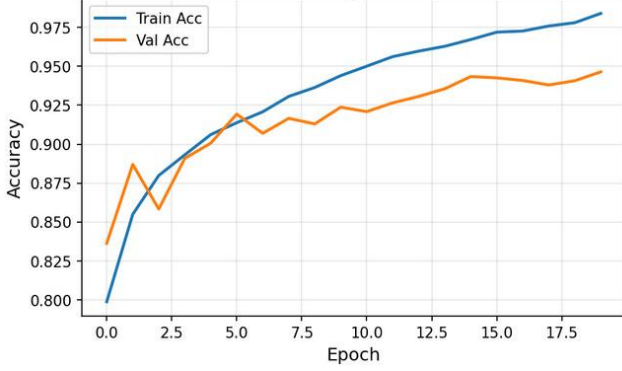


Figure 10: Training and validation accuracy curves for split boundary prediction

➤ Summary of Methodology

The developed framework by the decomposition of sandhi splitting into boundary detection and restoration manages to strike a balance among accuracy, interpretability, and computational feasibility. Apart from better error control, the two-stage design also leads to the failure cases' in-depth analysis, the results of which provide not only valuable linguistic but also computational insights into Sanskrit sandhi processing.

VII. ANALYSIS OF MODEL ERRORS AND FAILURE CASES

Neural sequence models have proven highly capable of learning and predicting Sandhi-related patterns from annotated examples. Since they can model character-level transformations, they are able to generalize to a large number of commonly occurring Sandhi cases. But the result outputs from these models when examined closely show that certain error groups are present across various architectures and experimental settings. Conducting a thorough investigation of such mistakes enlightens us of the limitations of current methods and also points out the aspects that have to be improved for the overall solution to become of a higher standard.

Firstly, an important set of errors in neural Sandhi splitters is related to the wrongly identified boundaries. Models frequently have problems deciding where the split should be when the surface form allows for more

than one linguistically valid segmentation point. It is especially the case with long compounds where several Sandhi changes could have been done one after the other. The more changes there are, the more the surface form will deviate from the original word boundaries after which it will become harder and harder for models to be sure of the exact split positions.

Phonological restoration - In fact, even if the model has located the boundary correctly, it may be unable to reconstruct the original words accurately. The most frequent types of restoration errors are vowel omission, vowel insertion where vowels are not present, wrong vowel length, and consonant substitution-as well as cases of assimilation or modification due to Sandhi when consonants are improperly handled. These errors imply that neural models are capable of understanding the general patterns of changes, yet, they apparently have difficulties with the details of phonology that are essential for accurate reconstruction. Such errors show that the neural models have only partially learned the phonological rules and have not completely internalized the linguistic principles.

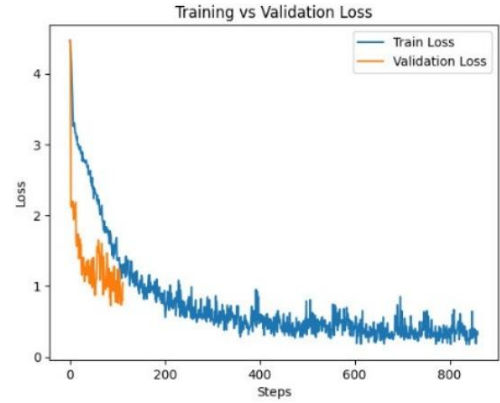


Figure 11: Training and validation loss curves for the Seq2Seq Sandhi splitting model

Ambiguity plays an additional role in worsening error behavior. Without a doubt, many of the sandhied forms can be divided in several ways, all of which are considered grammatically correct under classical Sanskrit guidelines. When there is no sentence-level or semantic context, neural models might have to depend only on the statistical regularities that they have learned from the training data to decide on one interpretation over another. Consequently, a predicted split might be linguistically justified but different from the gold-standard annotation. According to the standard evaluation protocols, such outputs are treated as errors even if they convey valid alternative analyses. This points to a shortcoming of purely word-level modeling and calls into question the evaluation strategies for tasks that are inherently ambiguous at the linguistic level.

Errors are very common when visarga and anusvāra transformations are involved. These types of Sandhi are extremely sensitive to the context, where the resultant phonological form depends on the most minute phonetic environments. Such patterns might be scarcely represented in the training data, especially in manually curated corpora of a limited size. As a result, models are often not able to generalize effectively in the case of these transformations, which can lead to either the wrong placing of the boundary or the restoration of the incorrect phonological form. The difficulty of these cases seems to emphasize not only the importance of having a context but also well-balanced data coverage.

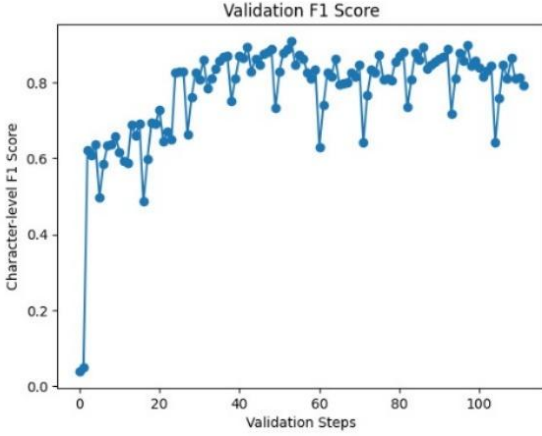


Figure 12 : Validation F1 score progression for the character-level Sandhi splitting seq2seq model.

The length of the compound is also of major importance in the accumulation of errors. With short compounds that only involve a single Sandhi transformation, neural models usually generate trustworthy and quite accurate outputs. On the other hand, long compounds tend to have the characteristic of a cascading accumulation of errors whereby a first mistake in boundary detection or restoration leads to the subsequent predictions getting an incorrect final result. This kind of behavior implies that Sandhi splitting models depend on their capacity to both take into account long-range dependencies and keep at the same time consistent character-level representations even over extended sequences. Those models that have not been intentionally designed for the purpose of handling such dependencies are especially prone to their performance going down as the length of the input increases.

In general, through error analysis, it is becoming clear that Sanskrit Sandhi splitting is still a highly complex problem even for sophisticated neural architectures. Current models seem to be able to cope well with frequently occurring and structurally simpler Sandhi cases but they still have serious problems with rare transformations, ambiguous forms, or long compounds.

These facts lead to the conclusion that there is the need for richer and more varied datasets, better modeling of phonological complexity, and perhaps the use of wider contextual information as well in the upcoming research. Taking on these issues is probably a prerequisite for making real advancements in the robustness and reliability of automated Sanskrit Sandhi splitting systems.

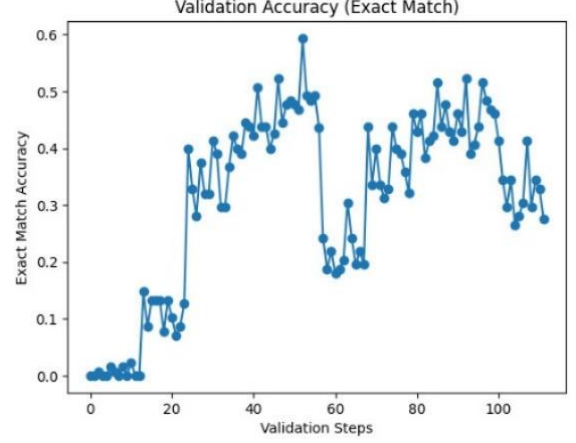


Figure 13 : Validation exact-match accuracy across training iterations for seq2seq model.

VIII. DISCUSSION

The experiments and observations in this report deal with numerous interesting aspects of applying neural models to the task of Sanskrit Sandhi splitting. This piece of work, by evaluating different neural architectures under a common preprocessing framework, enhances the understanding of how the different modeling decisions impact the capability of models to recognize word boundaries and restore the original phonological forms.

One important result is that end-to-end sequence-to-sequence neural networks can solve the problem of Sandhi splitting more efficiently than the boundary-only methods. These models which combine boundary detection and phonological restoration still capture better the intrinsic character of the Sanskrit Sandhi, because, character-wise, the transformations are the ones that change characters, insertions, or deletions of boundaries are rare. The authors here confirm that adopting the perspective of prior works, which argue that Sandhi splitting is better treated as a transformation of the original word rather than a mere segmentation, is valid.

A two-stage neural model also shows that dividing a problem into two steps of boundary localization and then restoration might increase the model's interpretability and stability. In the restoration part, the model gets a more focused task: its sole purpose

is phonological reconstruction. Thus, it is less disturbed by the global boundary decisions. Failure points will be clearer since the errors can be traced back to either incorrect boundary prediction or to restoration inaccuracies. Still, this advantage is gained at the cost of more complex architecture and higher interdependency between the steps.

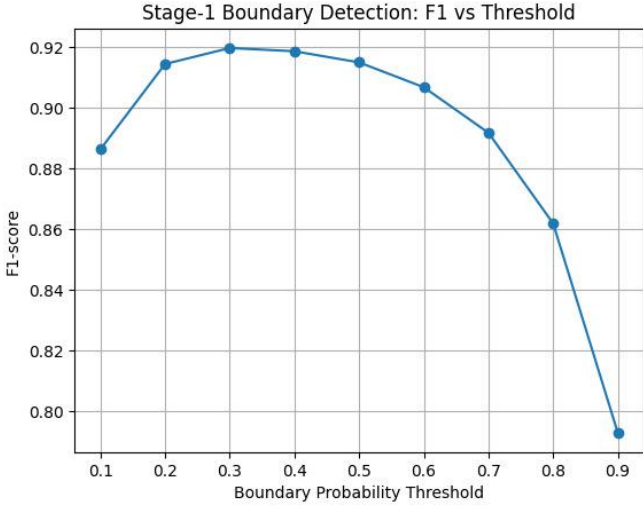


Figure 14: Boundary detection F1 score as a function of probability threshold

The other notable point in this discussion is about the length of compounds to be split. We have noticed a common behavior across all the models when it comes to the performance: as the sandhied compound grows longer, the performance degrades. All the difficulty comes from the fact that longer compounds usually entail multiple Sandhi transformations one after another.

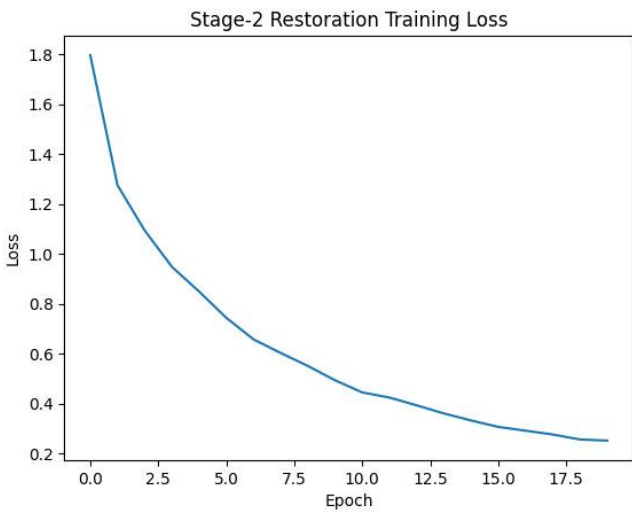


Figure 15: Stage-2 restoration training loss for the two-stage Sandhi splitting model

As a result, the challenges of both boundary detection and restoration escalate with the increase of the length. Here, the study's findings are in agreement with the prior reports, pointing to the same limitations of the recurrent models in their inability to capture long-distance dependencies, especially under the shortage of training data.

Moreover, the data suggests that some kinds of Sandhi are for sure those which are more straightforwardly handled as opposed to others. On the one hand, the neural models show a robust understanding of simple vowel combinations and frequently occurring transformations. On the other hand, they still struggle with rare or highly context-dependent transformations, for example, those involving visarga or ambiguous consonant changes. This means that the way data is distributed determines model performance to a large extent, and the less frequent Sandhi patterns are more error-prone.

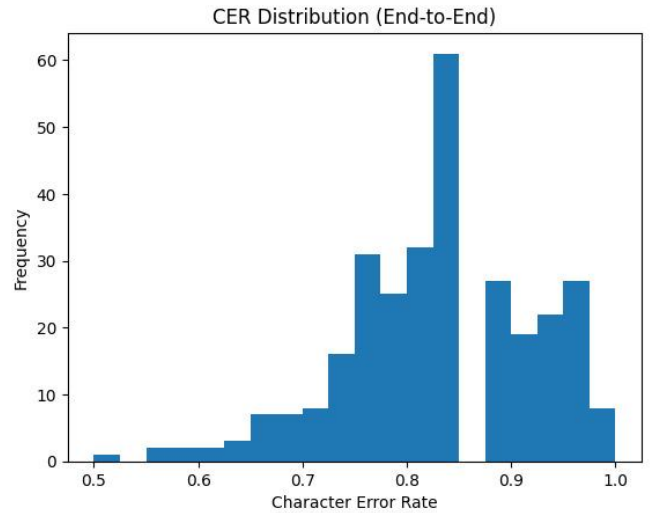


Figure 16: Distribution of character error rate (CER) for end-to-end Sandhi splitting

When these findings are contrasted with the trends disclosed in the related literature, the former most of the time fit with the latter. Extensive researches have been done to illustrate the significance of character-level modeling for the task of Sanskrit Sandhi splitting, and this study serves as a further proof of this argument. Nevertheless, a few standardized datasets and evaluation protocols remain largely unavailable, disallowing different studies to compare results directly, thus calling for more uniform experimental setups.

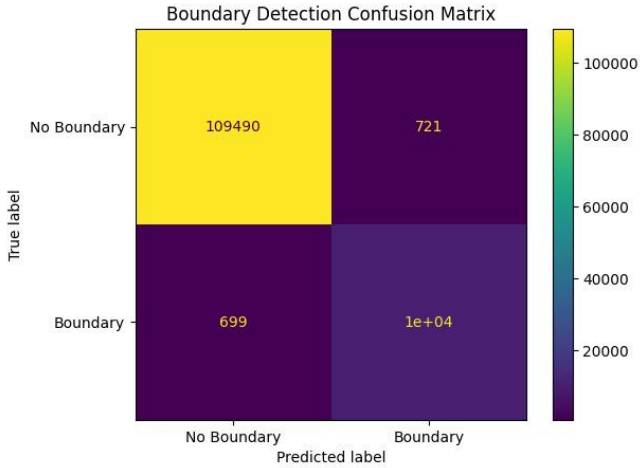


Figure 17: Confusion matrix for boundary detection in the two-stage model

In sum, the report suggests that neural models are a very good and versatile tool for the separation of Sanskrit Sandhis, yet it would be a mistake to consider them as a silver bullet. Their performance is highly correlated with the quality of the data, the complexity of the compounds, and the models' capabilities of capturing long-distance phonological dependencies. The takeaways here stress the significance of a strong neural backbone supported by richer and more informative data and contexts in the future endeavors.

IX. FUTURE SCOPE

The study described in this report is based on the use of neural sequence modeling techniques for Sanskrit Sandhi breaking at the word level. The models that were tested exhibit the capability to extract Sandhi-related regularities from the data; however, there are several natural directions which can be taken to enhance their robustness and widen their applicability.

An essential direction of incorporating context-aware Sandhi splitting should be considered. The current research work results in Sandhi breaking only on the words without sentence-level context which hinders the capacity to resolve the cases which are ambiguous and thus multiple valid splits can be found. Providing context not only from the surrounding words but also using models at the sentence-level can help to disambiguate such situations more effectively.

Another possible extension is using more extensive and diverse datasets. The expansion of the training set through the inclusion of varying Sandhi types, compound lengths, and text sources could be beneficial for the model in terms of generalization. The study of rare and complicated Sandhi phenomena which are scarcely present in small datasets could also become possible.

There is also the possibility of combining the joint modeling approaches, where Sandhi breaking is integrated numerically as a downstream task with, for example, morphological analysis or syntactic parsing. Such integrated models could achieve better performance through shared representations and they are less prone to error propagation as compared to pipeline models.

Lastly, further work on model interpretability and error analysis may yield valuable linguistic insights. Attending to the issue of why specific Sandhi transformations are really the ones constantly being mispredicted could in turn facilitate the creation of hybrid systems that may be based on neural learning coupled with linguistic constraints.

It is a basis for the construction of more advanced and context-aware Sanskrit NLP systems that the current work would constitute a starting point.

X. CONCLUSION

The study reported herein is a concentrated investigation into Sanskrit Sandhi segmentation using deep learning methods. In Sanskrit NLP, Sandhi splitting is a preparatory task that is quite challenging, for it involves both the precise determination of the word boundaries and the accurate phonological form recovery. The intricacy of the metamorphoses of Sandhi and the lack of explicit word boundaries stand it as a tough problem for computational models.

Here Sandhi splitting was treated as a character-based sequence modeling task. Different neural models, including a sequence-to-sequence BiLSTM network and a two-stage neural pipeline, were tried to see to what extent they can resolve boundary detection as well as phonological restoration. Thanks to the use of the same dataset format and preprocessing pipeline, the models' behaviors could be analyzed systematically.

To sum up, this work supports the claim that Sanskrit-Sandhi segmentation by means of neural sequence models is not only a feasible but also a very effective approach. Moreover, the report is also a reminder that more data, contextual modeling, and a higher level of linguistic integration are necessary to fully grasp the complexity of Sandhi. The current results will be helpful for future research on Sanskrit text processing based on neural approaches and provide a solid foundation for developing those techniques.

XI. REFERENCES

- [1] G. Huet, "The Sanskrit Heritage Engine," *Proc. Intl. Symp. on Sanskrit Computational Linguistics*, 2006.
- [2] A. Goyal and G. Huet, "Completeness analysis of a Sanskrit reader," *Sanskrit Computational Linguistics*, 2013.
- [3] A. Kulkarni et al., "Samsādhānī: A Sanskrit computational toolkit," *Proc. IJCNLP*, 2017.
- [4] V. Mittal, "Automatic Sanskrit segmentizer using finite-state transducers," *ACL Student Research Workshop*, 2010.
- [5] S. Bhardwaj et al., "SandhiKosh: A benchmark corpus for evaluating Sanskrit Sandhi tools," *LREC*, 2018.
- [6] O. Hellwig, "Using recurrent neural networks for Sanskrit Sandhi splitting," *LTC*, 2015.
- [7] A. Deshmukh and P. Joshi, "BiLSTM-CRF models for Sanskrit Sandhi splitting," *ACL*, 2021.
- [8] S. Kumar and P. Verma, "Morphological segmentation of Sanskrit using neural models," *COLING*, 2019.
- [9] V. Reddy et al., "Building a Sanskrit word segmenter overnight," *LREC*, 2018.
- [10] R. Aralikkatte et al., "Sanskrit Sandhi splitting using seq2(seq)²," *EMNLP*, 2018.
- [11] S. Dave et al., "Neural compound word generation and splitting in Sanskrit," *CODS-COMAD*, 2021.
- [12] O. Hellwig and S. Nehrlich, "Joint compound splitting and Sandhi resolution," *EMNLP*, 2018.
- [13] K. Bhatt et al., "CharSS: Character-level Transformer for Sanskrit word segmentation," *ACL Findings*, 2024.
- [14] J. Sandhan et al., "Transformer-based linguistically informed Sanskrit tokenization," *EMNLP*, 2022.
- [15] <https://sanskrit.uohyd.ac.in/Corpus/SHMT/Sandhi-Split/UOHYD/charakasamhita-sandhi-splitted-1-30-chapters.txt>